C-MCPN: The Clingo McCulloch-Pitts Network Editor

An Answer-Set Programming Framework for Artificial Neural Networks

David Gonzalez[1], Joshua T. Guerin, Ph.D.[2]
Mathematical Sciences
University of Northern Colorado
Greeley, CO 80639
[1]********@bears.unco.edu
[2]********@unco.edu

Abstract

As Artificial Intelligence and generative AI continue to engage the public in ways that mirror other critical advances (e.g., the PC and smartphone revolutions as well as the broader Internet) public interest has turned more and more towards applications of the underlying technologies and architectures that power them. While machine learning technologies like LLMs have a profound capacity to integrate AI into modern life, the underlying artificial neural network technologies present significant concern for both short and long-term sustainability: data center footprint, waste heat and noise, and infrastructure/resources for energy and clean water.

At the same time, advanced declarative languages have made remarkable headway towards AI solutions are considerably more efficient. In particular, Answer Set Programming languages are designed to solve computationally difficult (e.g., NP-Complete and NP-Hard) problems with limited available resources. In this paper we outline how these languages can be used to model artificial neural network architectures, like those used in LLMs, in the modeling language Clingo. We also introduce the C-MCPN (Clingo McCulloch-Pitts Network) Framework and Editor– a tool largely written in Python and Clingo. C-MCPN was developed to support our work in generating more efficient neural networks by providing a graphical interface to work with these models, using Clingo as a back-end for both representation and inference.

1 Introduction

Artificial Intelligence systems have had a significant impact on the modern world over the decades. Some of the many advancements across various fields include conversational agents, marketing, medicine, and insurance [7]. Unlike the impacts of AI in previous decades, general public awareness has skyrocked due to advances in generative AI (GEN-AI), in particular techniques from machine learning (ML) like the artificial neural networks (ANNs) that power contemporary large language models (LLMs).

While these AI systems have the potential for significant advancement, the underlying technologies come with significant drawbacks: footprints of data centers (which has garnered particular public interest–[6]), worsening use of finite resources such as fresh water and electricity, and the potential for significant environmental and health impacts [4].

Conversely, Answer Set Programming (ASP) languages often live at a different end of the research spectrum in terms of their efficiency. These languages are designed to solve NP-Complete and NP-Hard problems in a way that is fast and efficient on limited hardware such as consumer computers. Our current work involves exploring the use of ASP languages to accelerate the processes of learning and inference, with a hope of cutting into the resources needed for these technologies to continue growing in use.

In this paper we demonstrate that ANNs, specifically McCulloch-Pitts Networks (MPNs) can be used to model and conduct inference over McCulloch-Pitts neuroons [5]. We also introduce `C-MCPN`: The *Clingo McCulloch-Pitts Network Editor*. C-MCPN is a tool that we have developed to assist in the exploration of ASP-based networks, and allows the user to model network architectures, including the underlying representations and inference over ANNs.

2 Background

Our work is related to the merging of ANN technology with more efficient Answer Set Programming languges. In this section we will provide a brief background on both subjects, motivating our work on C-MCPN.

2.1 Artificial Neural Networks (ANNs)

Note: *As far as I can tell we are basically missing this section in the other paper(?), which we will need. In the other paper, section 2.3, there are a lot of citations and descriptions around the general subject, but nothing like "here is the definition of an ANN as an example."*

2.1.1 Environmental Impacts

The introduction of big data (in particular by scraping the broader Internet) changed the landscape for AI, allowing statistical models to flex their strength, in particular because these models require massive amounts of data for training purposes [**Helm2020**, 1]. This incredible need for information highlights an important drawback in the form of *data efficiency* [**Cropper2020**]. This notable lack of efficiency translates to a significant need in terms of resources.

As an example, let us consider a the cost to train a single model, ChatGPT-3. According to DeVries **(et al.? I can't find the bib right now.)**, the training of GPT-3 required around 1.3 million kWh *just* to be trained [9]. With some quick, back-of-the-envelope calculations, we can put this into perspective. Consider that the average American household uses approximately 10,791 kWh per year **is the source EIA2025?**. Dividing these out we find that the model's energy use was proportional to:

$$1{,}300{,}000 \text{ kWh} \times \frac{1 \text{ household}}{10{,}791 \text{ kWh/year}} \approx 120.5 \text{ households.}$$

Similarly, we could produce a rough, *lower bound* of how much water was likely used for training. A termoelectric power station can use approximately 43.8 L/kWh of water [**Gordon2024**]. That would put the *indirect* water costs of training GPT-3 at approximately 56,940,000 L, approximated by:

$$1{,}300{,}000 \text{ kWh} \times 43.8 \frac{\text{L}}{\text{kWh}} = 56{,}940{,}000 \text{ L}$$

Given that an adult male averages approximately 4L of fresh water, that number represents sufficient water to sustain a lower bound of approximately 39,000 adults for a year **Citations needed: 4L., possible arithmetic needed to spell out 39K**. To put this number into perspective, this would likely be sufficient to provide fresh water to approximately the city of Brighton, CO, for a year.

Note that this figure includes only the indirect costs to generate the electricity– massive cooling costs for data center equipment would likely add significantly to our calculation. While these calculations are remarkably overly-simplified, we hope that it puts into perspective how the technology is already unsustainable. This highlights the dire need for more efficient AI and ML technologies [9]. The discovery of models that are sufficiently sustainable to curb this resource use is an open problem.

We believe that one path towards a solution is to combine different AI methodologies, in particular those technologies that are known for higher efficiency. One possible candidate would be the use of satisfiability solvers, in

particular modern Answer Set Programming languages, to accelerate the learning and inference processes.

2.2 Answer Set Programming (ASP)

Answer Set Programming serves as an amalgamation of Prolog-style (i.e., quantified logic) with powerful advances in satisfiability (SAT) solvers that occurred throught the 1990s and into the 2000s, expanding significantly in terms of the model sizes they are capable of processing and computational efficiency. In this section we will discuss the basic syntax and semantics of the ASP language, Clingo, which serves as a technological foundation of C-MPCN [3, 2].

2.2.1 Syntax and Semantics

The atomic units in an ASP dialect such as Clingo are *literals*, which describe ideas and *facts* that serve as statements that are held to be true. The basic syntax is the application of a *property* to a literal, `property(atom)`. E.g.,

```
red(apple).
```

is a complete statement (and program), attributing a property of red to an idea named `apple`. Such statements are *purely declarative*, providing a *description* of the problem (vs. specific processor instructions). I.e., we describe *what* needs solving vs. *how* to solve the problem in traditional imperative/algorithmic contexts.

Further knowledge is*inferred* using aggregate statements known as *rules*, taking the form: `consequent :- antecedent`. This mirrors an *implication* from formal logic, as the `:-` symbol is read equivalently to $\leftarrow$. In such a statement the consequent *must* be true whenever the antecedent is true. E.g., consider the statement:

```
eat(Fruit) :- red(Fruit), ripe(Fruit).
```

This describes a preference over what fruit will be eaten. I.e., "If a fruit is red and ripe, then we eat the fruit."

In this case rather than the literal `apple`, we use a *variable* to allow for inference. I.e., The `Fruit` can be *any* contextually-relevant literal in the program, so long as it possesses the required properties. Note that this functions differently from its analogous notion in imperative language–rather than a pattern to be matched variables typically serve as typed memory locations that are manipulated by the program.

2.3 ANNs in Clingo

As an illustrative domain we have selected digital logic for hardware development. This domain has several attractive features including simplicity and that it is commonly taught in most CS programs. This also allows us to develop networks of arbitrary complexity for experimentation.

For our example network we used neurons with three types of well established logic gates: AND, OR, and XOR. In Table 1 we show the definitions of these gates, and their truth tables. Using a combination of these and more gates, one is able to construct complex logical operations[1]. Moreover, we believe that the C-MCPN Framework has the potential to be used for a wide range of applications in the future, especially if its capabilities are further explored and developed.

AND			**OR**			**XOR**		
$g(\vec{x}) = x_1 \cdot x_2,\ \theta = 1$			$g(\vec{x}) = x_1 + x_2,\ \theta = 1$			$g(\vec{x}) = \lvert x_1 - x_2 \rvert,\ \theta = 1$		
Activation: $f(\vec{x}) = \begin{cases} 0 & \text{if } g(\vec{x}) < \theta, \\ 1 & \text{if } g(\vec{x}) \geq \theta. \end{cases}$								
AND			OR			XOR		
x_1	x_2	$f(\vec{x})$	x_1	x_2	$f(\vec{x})$	x_1	x_2	$f(\vec{x})$
T	T	T	T	T	T	T	T	F
T	F	F	T	F	T	T	F	T
F	T	F	F	T	T	F	T	T
F	F	F	F	F	F	F	F	F

Table 1: McCulloch-Pitts Neuron Definitions and Truth Tables.

Did this table come from somewhere that needs citing, or did you come up with the layout and content?

The representation of such an artificial neuron requries both the *structure* designation, as well as the function used to *activate* the neuron. When activated the neuron will propagate a singnal to its output given the provided input signals.

We will use an `AND` gate to illustrate how we represent an artificial neuron:

```
% type("TYPE")
type("AND").
```

[1] An elegant and simple proof of Clingo's Turing completeness falls out of the C-MCPN Framework.

```
% gate(TYPE, OUTPUT, INPUTS)
gate("AND", Out, In1, In2) :- values(In1), values(In2),
                              Out = (In1 * In2).
```

In this case the *structure* of the neuron is indicated by the parameters: in the case of an `AND` gate this would require two input parameters and a single output. The *activation function* is encoded directly in the rule structure using the arithmetic equivalent provided in Table 1.

By specifying the *connectivity* of these neuron definitions, we can forge the connections of a network. In this case we will create a simple 1-bit adder circuit:

```
% neuron(type, unique_id)
neuron("INPUT", "INPUT_1").
neuron("XOR", "XOR_1").

% edge(source_id, target_id)
edge("INPUT_1", "XOR_1").

% There can be at most 2 inputs.
{ edge(src, "XOR_1") : neuron("XOR", "XOR_1") } 2.

% val(neuron_id, value).
val("INPUT_1", 1).
```

Note the *cardinality constraint*, enforcing at most two inputs. These constraints allow additional numeric constraints to be embedded in rules. Syntactically, this can be summarized as `l { } u`, where `l` and `u` prescribe (optional) upper and lower bounds on the contents.

The value inference program performs two important roles. It activates each gate by allowing the edges between neurons to carry values, and it assigns each individual neuron a value based on its gate type. The following lines are from the value inference program, and show how this is implemented:

```
%% Value domain
values(0;1).

% Edges carry values from input to output
edge(In, Neuron_id, Val) :-
    edge(In, Neuron_id),
    neuron(_, Neuron_id),
    neuron(_, In),
    val(In, Val).
```

```
% 2 input gates
val(Neuron_id, Val) :-
    gate(Type, Val, Val1, Val2),
    neuron(Type, Neuron_id),
    edge(In1, Neuron_id, Val1),
    edge(In2, Neuron_id, Val2),
    In1 != In2.

%% Each neuron has one value
{ val(Neuron_id, Val) : values(Val) } 1 :- neuron(_, Neuron_id).

%% Show values
#show val/2.
```

Note the first fact which sets the domain of values to be binary and the constraint that ensures each neuron has exactly one value. Most importantly, the rule which carrys the values along the edges is vital to the functioning of the network. Without it, the network gets "stuck" and is unable to infer what the values of the neurons should be. This is a key insight we had during development.

3 C-MCPN

In order to better explore the domains described in this paper we developed the C-MCPN software package to automate our workflow. The system itself is designed around a standard (unix-like) terminal use and is extended into a graphical user interface (GUI).

The core functionalities of C-MCPN are built on the ASP language Clingo, as described in the preceding sections. The editor and interface were constructed in Python using the DearPyGui library, allowing us to take advantage of its node-based editing features.

Building the editor in this way also allowed us to easily integrate it with the C-MCPN Framework, as we could directly use the same two-layered structure for elaborating both the value of each neuron as well as the network architecture. This was an important design choice, as it allows the editor to be easily extended in the future to support a wider range of applications.

In terms of the reorganization algorithm we used a modified version of the Sugiyama algorithm, taking just the layer assignment step and a simplified node positioning step [8].

3.1 C-MCPN Framework

From what we could find in the literature, the C-MCPN Framework is a novel approach to neural network representation within a logical programming paradigm like ASP. The networks themselves have relatively simple representations that can be leveraged to create complex operations. The framework, which can be seen in Figure 1, uses three main program types:

- **Gate Definitions Program:** Files are in the `C-MCPN/gates/` directory.
- **Network Structure Program:** Files are in the `C-MCPN/networks/` directory.
- **Value Inference Program:** Files are in the `C-MCPN/base/` directory.

The files in the gate definitions program define the behavior of each gate, including the number of inputs it should have. Every default gate we included also has a special header used to send meta-information to the C-MCPN Editor. (E.g., The `AND` formulation specified in Section 2.3.

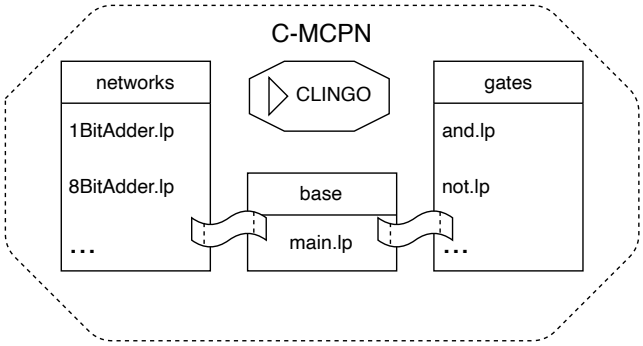

Figure 1: A visualization of the programs involved in the C-MCPN Framework.

3.2 C-MCPN Editor

In the `app/` directory of the project, we have the code for the C-MCPN Editor. The main features of the editor include the ability to add/remove neurons, connect/disconnect them, copy/paste them, and run inference on the network. The editor allows the files to be edited directly from the interface, transforming it into more of an interactive development environment (IDE) for logical

networks. It includes a preview feature which allows users to see the structure of the network in ASP format before and after running the inference with simple syntax highlighting; helpful for debugging and understanding how the network is functioning. There is also an option to create new logic gates that the neurons can use, allowing for a wider range of applications.

Custom networks are saveable to JSON or ASP format and loadable from JSON format, making it easy to work on them over multiple sessions and share them with others. Saving the network to ASP format allows direct use with the C-MCPN Framework from the terminal instead of IDE.

The inputs for the inference use a Python script, allowing users to create complex inputs for their custom networks. The output can also be parsed with savable Python scripts built directly in the editor, allowing the user to choose how they want to process the results. Finally, it is worth noting that the editor includes a reorganization algorithm which automatically rearranges the neurons to make the network easier to read and understand after changes are made. This algorithm is based on the Sugiyama algorithm for layered graph drawing [8].

A screenshot of the editor can be seen in Figure 2, showing a sample 1-bit adder network.

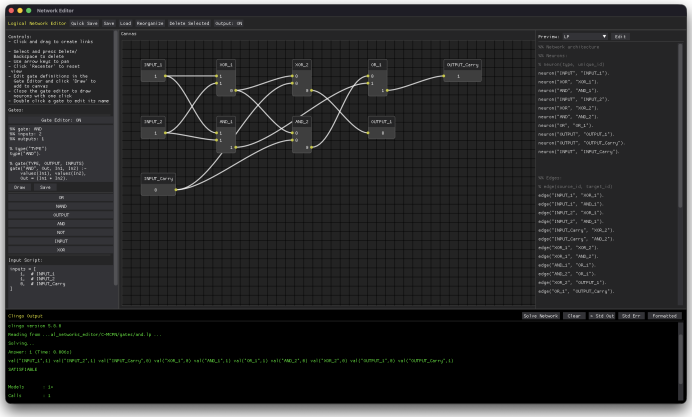

Figure 2: A screenshot of the C-MCPN Editor, showing a sample 1-bit adder network.

3.3 AI-Assisted Development

Claude (Anthropic) was used as a programming assistant throughout development of the editor, helping with DearPyGui module integration, debugging, and some code generation. All design decisions, research direction, and domain-specific logic (ASP, gate definitions, network structure) were conscious decisions by the authors. Each line of code generated by Claude was reviewed, edited, and approved by the authors before being added to the codebase. In order to maintain a supervisory-level of control the agent was not given direct access to the codebase or the repository. The file that Claude had input in are contained in the `app/` directory of the project, allowing users the ability to inspect these files if they so desire.

Use of Claude was employed only in the above described sections in order to accelerate the development process under circumstances of limited resources.[2]

4 Evaluation

Once the screenshot is removed this section may be something we can kill for space. The 1-bit adder example network was used to evaluate the framework for correctness. Its truth table was compared to the expected output for all permutations of possible valid inputs. The verified network can be seen in in Figure 3.

The C-MCPN Framework works well for logical operations with a fixed number of inputs and outputs, and the C-MCPN Editor makes it easy to construct and visualize these networks. There were two key insights gained throughout the creation of this framework which are the importance of the edges carrying values and the minimalistic fact structure which improves the performance of the network. This framework is a novel approach to neural network representation within a logical programming paradigm like ASP.

The C-MCPN Editor also shows potential as a powerful educational tool for understanding both neural networks and logic programming. Showing a visualization of the network structure along with a live preview of the code necessary to create it is a great way to help users relate the two ideas.

Editor screenshots should be moved to the preceding section.

4.1 Limitations

This work is a proof of concept for the C-MCPN Framework and Editor, and as such there are some limitations to be aware of. The framework has no cycle detection nor has it been tested on recurrent networks. This is likely

[2]The authorship of this paper is without the assistance of any AI system, including planning, writing, editing, and repository maintenance.

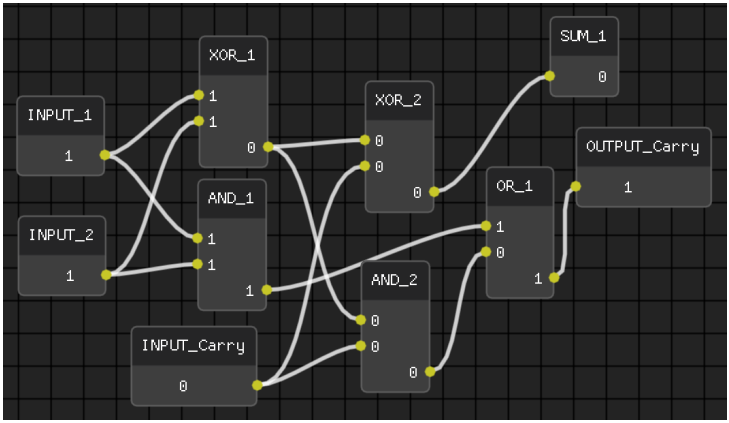

Figure 3: A screenshot of a 1-bit adder network in the C-MCPN Editor.

to cause issues with the value inference program, as it relies on the values being propagated along the edges. The framework has also not been tested on large networks, and it is unknown how scale will affect its performance. It is also worth noting that Clingo does not have native support for floating point numbers, limiting the potential applications of easy to understand networks.

5 Conclusion

The C-MCPN Framework and Editor provide a novel approach to representing and working with logical networks within the paradigm of logic programming. The framework allows for the construction of logical networks using simple facts and rules, while the editor provides an interactive environment for building and visualizing these networks. Full instructions for using the editor can be found in the README of the project, and we encourage readers to try it out for themselves at `https://github.com/David-Gonzalez-Sua/logical_networks`. Importantly, the editor is designed with accessibility in mind, and we hope it can be used by a wide range of people to learn about these concepts.

5.1 Future Work

There is a lot of potential for future work in this area, both in terms of improving the C-MCPN Framework and Editor, and in exploring new applications for logical networks. For example, we have begun work on a new framework for an adaptive neural network which can learn on data that uses the C-MCPN

Framework as its skeleton. This framework is in its early stages, but we hope to also add it as a feature in the editor in the future.

References

[1] Bikram Pratim Bhuyan et al. "Neuro-symbolic artificial intelligence: a survey". In: *Neural Computing and Applications* 36.21 (2024), pp. 12809–12844. DOI: http://dx.doi.org/10.1007/s00521-024-09960-z.

[2] Pedro Cabalar et al. "On the Semantics of Hybrid ASP Systems Based on Clingo". In: *Algorithms* 16.4 (2023), p. 185. DOI: http://dx.doi.org/10.3390/a16040185.

[3] Martin Gebser et al. "Multi-shot ASP solving with clingo". In: *Theory and Practice of Logic Programming* 19.1 (2019), pp. 27–82. DOI: http://dx.doi.org/10.1017/S1471068418000054.

[4] Yuelin Han et al. *The Unpaid Toll: Quantifying and Addressing the Public Health Impact of Data Centers*. http://arxiv.org/abs/2412.06288. 2025.

[5] Warren S. McCulloch and Walter Pitts. "A logical calculus of the ideas immanent in nervous activity". In: *The Bulletin of Mathematical Biophysics* 5.4 (1943), pp. 115–133. DOI: http://dx.doi.org/10.1007/BF02478259.

[6] Engineering National Academies of Sciences and Medicine. *Implications of Artificial Intelligence–Related Data Center Electricity Use and Emissions: Proceedings of a Workshop*. Ed. by Anne Frances Johnson and Kasia Kornecki. Washington, DC: The National Academies Press, 2025. DOI: http://dx.doi.org/10.17226/29101.

[7] Albert Nössig, Tobias Hell, and Georg Moser. "Rule learning by modularity". In: *Machine Learning* 113.10 (2024), pp. 7479–7508. DOI: http://dx.doi.org/10.1007/s10994-024-06556-5.

[8] Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiko Toda. "Methods for Visual Understanding of Hierarchical System Structures". In: *IEEE Transactions on Systems, Man, and Cybernetics* 11.2 (1981), pp. 109–125. DOI: http://dx.doi.org/10.1109/TSMC.1981.4308636.

[9] Alex De Vries. "The growing energy footprint of artificial intelligence". In: *Joule* 7.10 (2023), pp. 2191–2194. DOI: http://dx.doi.org/10.1016/j.joule.2023.09.004.